CAPSTONE PROJECT:

MediLink Health Network Enterprise Data Platform

One project, built phase-by-phase across 15 weeks. Every course module becomes a build step. Teams of 3–5 students (rotate roles: Data Steward, Custodian, Analyst, Documenter). All code is MySQL 8.0+.

1. THE BUSINESS PROBLEM

MediLink is a clinic network in Uganda formed by merging 4 previously independent clinics (Kampala, Mukono, Mbarara, Gulu). Each came with its own patient registration system; one branch still uses paper. The consequences:

- ✓ "Sarah Nakato" exists 3 times across systems with different spellings and phones nobody knows how many real patients the network has.
- ✓ A nurse once emailed a spreadsheet containing patient HIV statuses to the wrong person a near-violation of the Uganda Data Protection and Privacy Act.
- ✓ The Ministry of Health requires a consolidated monthly report (OPD attendance, top diagnoses, revenue by service) that currently takes 3 staff 2 weeks to compile by hand.
- ✓ No one knows who owns which data, which numbers are trustworthy, or what can be deleted.

Your job: as the newly hired EDM team, build the entire enterprise data platform governance, clean master data, a data warehouse, metadata, security, and reporting in one semester.

2. BUSINESS RULES (your requirements document)

- ✓ Every patient has exactly one master (golden) record, no matter how many branch registrations exist.
- ✓ Phone (normalized to `256XXXXXXXXX`) is the primary match key; National ID is stored as a SHA-256 hash only, never plain text.

- ✓ A visit belongs to exactly one clinic, one patient, one staff member; billing rows are immutable (corrections = adjustment rows, never edits).
- ✓ Survivorship rule: contact fields (phone, email) most recently updated source wins; identity fields (name, DOB, sex) never auto-merge, conflicts go to a steward review queue.
- ✓ Master data quality target: ≥97% phone completeness, 0 invalid phones measured weekly and logged.
- ✓ Access by role: Reception → masked directory only; Clinician → full record; Analyst → de-identified warehouse only; every change to the master is audited.
- ✓ Clinical records retained 10 years → archived → purged (automated lifecycle).
- ✓ MoH report (attendance, top-10 diagnoses, revenue by service) delivered by the 5th of each month.
- ✓ Every source system must register metadata (owner, definitions, lineage) in the catalog before it is trusted.

PHASE 0 (Week 1): Setup & Source Inventory

- ✓ Install MySQL 8 + Workbench; create Git repo `medilink-edm`.
- ✓ Walk through (role-play or visit) one real health facility / SACCO / pharmacy and list its data sources.

```sql
CREATE DATABASE medilink CHARACTER SET utf8mb4;

USE medilink;
```

Deliverable: source inventory table for each system: name, owner, format, volume, quality reputation.

PHASE 1 (Weeks 1–2): Data Lifecycle Design

Key decisions: for each data class, define create → store → use → archive → delete.

```sql
-- Lifecycle control table (the register for Rule 7)
CREATE TABLE data_lifecycle_policy (
    data_class    VARCHAR(50) PRIMARY KEY,          -- e.g. CLINICAL, BILLING
    retention_years INT NOT NULL,
    archive_after_years INT NOT NULL,
    lawful_basis VARCHAR(100),                      -- Uganda DPA: consent / care / legal duty
    owner         VARCHAR(100)
);

INSERT INTO data_lifecycle_policy VALUES
('CLINICAL', 10, 7, 'Provision of health care (DPA s.7)', 'Head of Clinical Services'),
('BILLING',   7, 5, 'Legal obligation', 'Finance Manager'),
('MARKETING_CONSENT', 2, 1, 'Consent', 'Data Steward');
```

Deliverable: 1-page lifecycle map per data class (feeds the governance document).

PHASE 2 (Weeks 3–4): Governance Framework Assessment 1 (20%)

Build in MySQL what the framework document describes:

```sql
-- 1. Accountability register (who owns what)
```

```
CREATE TABLE data_ownership (
    object_name VARCHAR(64) PRIMARY KEY,
    data_owner  VARCHAR(100),
    data_steward VARCHAR(100),
    custodian   VARCHAR(100)
);
INSERT INTO data_ownership VALUES
('m_patient','Medical Director','D. Nakato (Steward)','IT Custodian'),
('billing','Finance Manager','J. Okello (Steward)','IT Custodian');

-- 2. RACI matrix for key activities
CREATE TABLE raci (
    activity VARCHAR(60),
    cdo ENUM('R','A','C','I',''),
    steward ENUM('R','A','C','I',''),
    custodian ENUM('R','A','C','I',''),
    clinician ENUM('R','A','C','I',''),
    PRIMARY KEY(activity)
);
INSERT INTO raci VALUES
('Approve new source system','A','R','C','I'),
('Resolve duplicate patient','I','A/R','C','C'),
('Grant data access','I','C','A/R','I');
```

```
-- 3. Policy register
CREATE TABLE policies (
    policy_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100),
    rule_summary VARCHAR(300),
    enforcement ENUM('MANUAL','TECHNICAL') NOT NULL
);
INSERT INTO policies VALUES
(NULL,'Patient Identity Policy','One golden record per patient; steward reviews identity merges','TECHNICAL'),
(NULL,'Access Control Policy','Least privilege by role; PII masked outside clinical use','TECHNICAL'),
(NULL,'Retention Policy','Per data_lifecycle_policy table','TECHNICAL');

-- 4. Compliance gap register (Uganda DPA / GDPR mapping)
CREATE TABLE compliance_register (
    req_id INT AUTO_INCREMENT PRIMARY KEY,
    requirement VARCHAR(200),          -- e.g. 'Right to erasure (GDPR Art.17)'
    current_state VARCHAR(200),
    gap ENUM('CLOSED','PARTIAL','OPEN'),
    remediation VARCHAR(200)
);
INSERT INTO compliance_register VALUES
(NULL,'National ID not stored in plain text','Plaintext in Legacy A','OPEN','Hash at ingestion (Phase 8)'),
```

(NULL,'Breach notification within 72h','No procedure','OPEN','Breach playbook (Phase 8)');

```

***Deliverable: Governance Framework document (charter, roles, RACI, 3 policies, compliance map) Assessment 1.***

***PHASE 3 (Weeks 4–5): Architecture & Master Schema***

Architecture decision: hub-and-spoke a central master DB (hub), source systems (spokes) feed it via staging. Justify vs centralized/decentralized in one page.

```sql
-- Clinics & staff
CREATE TABLE clinics (
  clinic_id INT AUTO_INCREMENT PRIMARY KEY,
  clinic_code VARCHAR(10) UNIQUE,
  clinic_name VARCHAR(100), district VARCHAR(60)
);
INSERT INTO clinics VALUES
(NULL,'MP-KLA','MediLink Kampala','Kampala'),
(NULL,'MP-MUK','MediLink Mukono','Mukono'),
(NULL,'MP-MBR','MediLink Mbarara','Mbarara'),
(NULL,'MP-GUL','MediLink Gulu','Gulu');

CREATE TABLE staff (
  staff_id INT AUTO_INCREMENT PRIMARY KEY,

```
  full_name   VARCHAR(100),   role
ENUM('DOCTOR','NURSE','LAB','PHARMACIST','ADMIN'),
  clinic_id INT, FOREIGN KEY (clinic_id) REFERENCES clinics(clinic_id)
);

-- THE PATIENT MASTER (destination of MDM in Phase 5)
CREATE TABLE m_patient (
  patient_key INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  phone VARCHAR(13),   -- normalized 256XXXXXXXXX
  email VARCHAR(120),
  sex ENUM('M','F'),
  date_of_birth DATE,
  national_id_hash CHAR(64),   -- NEVER plaintext (Rule 2)
  blood_group ENUM('A+','A-','B+','B-','O+','O-','AB+','AB-'),
  home_district VARCHAR(60),
  status ENUM('ACTIVE','ARCHIVED') DEFAULT 'ACTIVE',
  source_count TINYINT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_phone CHECK (phone REGEXP '^256[0-9]{9}$')   -- validate at entry
);

-- Crosswalk: which source records belong to which master
CREATE TABLE patient_source_links (
  link_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  patient_key INT NOT NULL,
  source_system   ENUM('LEGACY_A','LEGACY_B','PHARMACY_POS','PAPER_SCAN') NOT NULL,
  source_record_id VARCHAR(50) NOT NULL,
  matched_on VARCHAR(30),
  confidence DECIMAL(4,3),
  UNIQUE (source_system, source_record_id),
  FOREIGN KEY (patient_key) REFERENCES m_patient(patient_key)
);

-- Operational tables
CREATE TABLE visits (
  visit_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  patient_key INT NOT NULL, clinic_id INT NOT NULL, staff_id INT,
  visit_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  visit_type ENUM('OPD','ANC','IMMUNIZATION','ADMISSION','LAB') NOT NULL,
  diagnosis_code VARCHAR(10), diagnosis_note VARCHAR(200),
  FOREIGN KEY (patient_key) REFERENCES m_patient(patient_key),
  FOREIGN KEY (clinic_id) REFERENCES clinics(clinic_id),
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id)
);

CREATE TABLE billing (
  bill_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  visit_id BIGINT NOT NULL,
```

```
  service
ENUM('CONSULTATION','LAB','MEDICATION','ADMISSION','PROCEDURE'),
  amount DECIMAL(12,2) CHECK (amount >= 0),
  payment_mode ENUM('CASH','MOBILE_MONEY','INSURANCE'),
  FOREIGN KEY (visit_id) REFERENCES visits(visit_id)
);

-- Normalization note for your report: m_patient is 3NF
-- (patient_key → all attributes; no repeating groups; no transitive deps).
```

Deliverable: architecture diagram (draw.io) + ERD + `schema.sql`.

PHASE 4 (Weeks 5–6): Data Quality Assessment 2 (15%)

The dirty data arrives (this seed simulates the legacy extract make it messier if you like):

```sql
CREATE TABLE stg_legacy_a (          -- old clinic database (messy)
  src_id INT PRIMARY KEY,
  pname VARCHAR(120), phone_no VARCHAR(25), gender VARCHAR(10),
  dob_raw VARCHAR(25), nid VARCHAR(25), last_visit DATE
);
INSERT INTO stg_legacy_a VALUES
(1,'nakato sarah','0772 100 001','F','1990-05-12','CM90123456ABC','2024-11-02'),
(2,'NAKATO SARAH','256772100001','Female','12/05/1990','','2025-01-15'),
(3,'Okello John  ','+256772100002','M','1985-3-3','CM85112233','2024-12-20'),
```

(4,'acheb gloria','0771003003','f','','','2025-02-01'),

(5,'Mwesigwa D','0782 455 901','male','1978/09/22','CM78998877','2024-10-30'),

(6,'Unknown Child','0','M','','','2025-02-03');

CREATE TABLE stg_legacy_b (-- online booking (cleaner, has email)

  src_id INT PRIMARY KEY, name VARCHAR(120), phone VARCHAR(25),

  email VARCHAR(120), sex VARCHAR(10), dob DATE, updated_at DATE

);

INSERT INTO stg_legacy_b VALUES

(101,'Sarah Nakato','256772100001','s.nakato@gmail.com','F','1990-05-12','2025-03-01'),

(102,'J. Okello','0772100002','okello.j@outlook.com','M','1985-03-03','2025-02-10'),

(103,'Gloria Acheb','256771003003',NULL,'F',NULL,'2025-01-20');

```

STEP 1  PROFILE (measure, with numbers):

```sql

SELECT

  ROUND(100SUM(nid IS NOT NULL AND nid<>'')/COUNT(),1) AS nid_complete_pct,

  ROUND(100SUM(gender IS NOT NULL AND TRIM(gender)<>'')/COUNT(),1) AS gender_complete_pct,

  COUNT() AS total_rows

FROM stg_legacy_a;

SELECT COUNT() AS invalid_phones FROM stg_legacy_a

WHERE phone_no NOT REGEXP '^(0|\\+?256)?[0-9]{9}$' OR LENGTH(REGEXP_REPLACE(phone_no,'[^0-9]','')) < 10;

SELECT DISTINCT gender FROM stg_legacy_a;  -- consistency: 'M','Male','f','female'... 6 values, 1 domain!

```

STEP 2 CLEANSE (systematic fixes into clean columns):

```sql

ALTER TABLE stg_legacy_a

 ADD COLUMN name_clean VARCHAR(120), ADD COLUMN phone_norm VARCHAR(13),

 ADD COLUMN sex_clean ENUM('M','F'), ADD COLUMN dob_clean DATE;

-- Names: Title Case

UPDATE stg_legacy_a

SET name_clean = CONCAT(UPPER(SUBSTRING(TRIM(pname),1,1)),

                LOWER(SUBSTRING(TRIM(pname),2)));

-- Phones: strip non-digits, force 256XXXXXXXXX

UPDATE stg_legacy_a

SET phone_norm = CASE

 WHEN REGEXP_REPLACE(phone_no,'[^0-9]','') LIKE '256%' AND LENGTH(REGEXP_REPLACE(phone_no,'[^0-9]',''))=12

```
      THEN REGEXP_REPLACE(phone_no,'[^0-9]','')
  WHEN LENGTH(REGEXP_REPLACE(phone_no,'[^0-9]',''))=10
      THEN CONCAT('256', SUBSTRING(REGEXP_REPLACE(phone_no,'[^0-9]',''),2))
  ELSE NULL END;   -- '0' becomes NULL → flagged, not guessed

-- Sex: consolidate the domain
UPDATE stg_legacy_a SET sex_clean = CASE UPPER(TRIM(gender))
  WHEN 'M' THEN 'M' WHEN 'MALE' THEN 'M' WHEN 'F' THEN 'F' WHEN 'FEMALE' THEN 'F' END;

-- Dates: try ISO, then D/M/Y
UPDATE stg_legacy_a SET dob_clean = STR_TO_DATE(dob_raw,'%Y-%m-%d') WHERE dob_raw REGEXP '^[0-9]{4}-';
UPDATE stg_legacy_a SET dob_clean = STR_TO_DATE(dob_raw,'%d/%m/%Y')
WHERE dob_clean IS NULL AND dob_raw REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$';
```

STEP 3 PREVENT (constraints on the master already added in Phase 3: `chk_phone`, ENUM domains).

STEP 4 MONITOR (the weekly scorecard Rule 5):

```sql
CREATE VIEW v_dq_scorecard AS
SELECT
  ROUND(100SUM(phone IS NOT NULL)/COUNT(),1)      AS phone_complete_pct,
  ROUND(100SUM(date_of_birth IS NOT NULL)/COUNT(),1) AS dob_complete_pct,
```

ROUND(100SUM(sex IS NOT NULL)/COUNT(),1) AS sex_complete_pct,

COUNT() - COUNT(DISTINCT CONCAT(full_name, IFNULL(phone,'x'))) AS potential_dupes

FROM m_patient;

SELECT FROM v_dq_scorecard;

```

***Deliverable: Data Quality Report before/after metrics per dimension, root-cause (fishbone: why did Legacy A store phones as text?), monitoring plan Assessment 2.***

***PHASE 5 (Weeks 7–8): Master Data Management Assessment 3 (15%)***

Goal: Sarah Nakato (A: rows 1&2, B: row 101) becomes one golden record with a survivorship decision for every field.

```sql

-- Normalize source B too (same clean columns)

ALTER TABLE stg_legacy_b ADD COLUMN phone_norm VARCHAR(13), ADD COLUMN name_clean VARCHAR(120);

UPDATE stg_legacy_b

SET phone_norm = CONCAT('256', SUBSTRING(REGEXP_REPLACE(phone,'[^0-9]',''),-9)),

name_clean = CONCAT(UPPER(SUBSTRING(TRIM(name),1,1)), LOWER(SUBSTRING(TRIM(name),2)));

-- Unified clean source view

CREATE VIEW v_all_sources AS

SELECT 'LEGACY_A' src_system, src_id, phone_norm, name_clean, sex_clean AS sex,

```
    dob_clean AS dob, NULL AS email, last_visit AS updated_at
FROM stg_legacy_a WHERE phone_norm IS NOT NULL
UNION ALL
SELECT 'LEGACY_B', src_id, phone_norm, name_clean, NULL, dob, email, updated_at
FROM stg_legacy_b WHERE phone_norm IS NOT NULL;

-- GOLDEN RECORDS: match on phone; survivorship = most recently updated row wins
INSERT INTO m_patient (full_name, phone, sex, date_of_birth, email, source_count)
WITH ranked AS (
  SELECT s.*,
    ROW_NUMBER() OVER (PARTITION BY phone_norm ORDER BY updated_at DESC) AS rn,
    COUNT() OVER (PARTITION BY phone_norm) AS n_src
  FROM v_all_sources s
)
SELECT name_clean, phone_norm, sex, dob, email, n_src
FROM ranked WHERE rn = 1;

-- Crosswalk: link every source row to its golden record (Rule 1)
INSERT INTO patient_source_links (patient_key, source_system, source_record_id, matched_on, confidence)
SELECT m.patient_key, s.src_system, s.src_id, 'PHONE', 0.95
FROM v_all_sources s JOIN m_patient m ON m.phone = s.phone_norm;

-- Unmatched / conflict records go to the STEWARD REVIEW QUEUE (Rule 4)
```

CREATE TABLE mdm_review_queue (

queue_id INT AUTO_INCREMENT PRIMARY KEY,

src_system VARCHAR(20), src_id INT, reason VARCHAR(60),

raised_at DATETIME DEFAULT CURRENT_TIMESTAMP,

resolved_by VARCHAR(60) NULL, resolved_at DATETIME NULL

);

INSERT INTO mdm_review_queue (src_system, src_id, reason)

SELECT 'LEGACY_A', src_id, 'NO_PHONE_CANNOT_MATCH' FROM stg_legacy_a WHERE phone_norm IS NULL;

```

Verify the merge:

```sql

SELECT m.patient_key, m.full_name, m.phone, m.email, m.source_count,

GROUP_CONCAT(l.source_system) AS found_in

FROM m_patient m JOIN patient_source_links l USING (patient_key)

GROUP BY m.patient_key;

-- Sarah: 1 master record, source_count=3, found_in=LEGACY_A,LEGACY_A,LEGACY_B ✓

```

Deliverable: MDM Implementation Plan architecture style (consolidation), matching rules, survivorship rules table (field-by-field justification), review workflow Assessment 3.

PHASE 6 (Weeks 8–9): Data Integration ETL into the Warehouse
```

Staging (already exists) → Transform → Load into a star schema.

```sql
-- Dimension tables
CREATE TABLE dim_clinic (
  clinic_sk INT AUTO_INCREMENT PRIMARY KEY,
  clinic_id INT UNIQUE, clinic_code VARCHAR(10), clinic_name VARCHAR(100), district VARCHAR(60)
);
INSERT INTO dim_clinic (clinic_id, clinic_code, clinic_name, district)
SELECT clinic_id, clinic_code, clinic_name, district FROM clinics;

CREATE TABLE dim_patient (
  patient_sk INT AUTO_INCREMENT PRIMARY KEY,
  patient_key INT UNIQUE, full_name VARCHAR(120), sex CHAR(1),
  age_band VARCHAR(10), home_district VARCHAR(60)
);
INSERT INTO dim_patient (patient_key, full_name, sex, age_band, home_district)
SELECT patient_key, full_name, sex,
      CASE WHEN date_of_birth IS NULL THEN 'UNKNOWN'
          WHEN TIMESTAMPDIFF(YEAR,date_of_birth,CURDATE()) < 5  THEN '0-4'
          WHEN TIMESTAMPDIFF(YEAR,date_of_birth,CURDATE()) < 18 THEN '5-17'
          WHEN TIMESTAMPDIFF(YEAR,date_of_birth,CURDATE()) < 60 THEN '18-59'
          ELSE '60+' END,
      home_district
FROM m_patient WHERE status='ACTIVE';
```

```
CREATE TABLE dim_date (
  d DATE PRIMARY KEY, yr SMALLINT, qtr TINYINT, mo TINYINT,
  mo_name VARCHAR(12), dow VARCHAR(10)
);
DELIMITER //
CREATE PROCEDURE fill_dim_date(IN start_d DATE, IN days INT)
BEGIN
  DECLARE i INT DEFAULT 0;
  WHILE i < days DO
    INSERT IGNORE INTO dim_date
    SELECT DATE_ADD(start_d, INTERVAL i DAY), YEAR(start_d + INTERVAL i DAY),
           QUARTER(start_d + INTERVAL i DAY), MONTH(start_d + INTERVAL i DAY),
           MONTHNAME(start_d + INTERVAL i DAY), DAYNAME(start_d + INTERVAL i
DAY);
    SET i = i + 1;
  END WHILE;
END //
DELIMITER ;
CALL fill_dim_date('2024-01-01', 800);

-- Fact table: one row per visit, amounts pivoted from billing
CREATE TABLE fact_visits (
  visit_sk BIGINT AUTO_INCREMENT PRIMARY KEY,
  visit_date DATE NOT NULL,
```

```
  clinic_sk INT NOT NULL, patient_sk INT NOT NULL,
  visit_type VARCHAR(20), diagnosis_code VARCHAR(10),
  consult_amount DECIMAL(10,2), lab_amount DECIMAL(10,2),
  med_amount DECIMAL(10,2), total_amount DECIMAL(10,2),
  KEY idx_fv_date (visit_date), KEY idx_fv_clinic (clinic_sk)
);

-- TRANSFORM + LOAD
INSERT INTO fact_visits
  (visit_date, clinic_sk, patient_sk, visit_type, diagnosis_code,
   consult_amount, lab_amount, med_amount, total_amount)
SELECT DATE(v.visit_date), dc.clinic_sk, dp.patient_sk, v.visit_type, v.diagnosis_code,
      COALESCE(SUM(CASE WHEN b.service='CONSULTATION' THEN b.amount END),0),
      COALESCE(SUM(CASE WHEN b.service='LAB'          THEN b.amount END),0),
      COALESCE(SUM(CASE WHEN b.service='MEDICATION'   THEN b.amount END),0),
      COALESCE(SUM(b.amount),0)
FROM visits v
JOIN dim_clinic  dc ON dc.clinic_id  = v.clinic_id
JOIN dim_patient dp ON dp.patient_key = v.patient_key
LEFT JOIN billing b ON b.visit_id = v.visit_id
GROUP BY v.visit_id, DATE(v.visit_date), dc.clinic_sk, dp.patient_sk,
         v.visit_type, v.diagnosis_code;
```

Document the ETL workflow (extraction method, transformation rules, load sequence, error handling) one diagram + table.

Deliverable: ETL workflow design + populated warehouse feeds Assessment 5 (10%).

PHASE 7 (Weeks 9–10): Metadata Management

```sql
-- 1. Technical metadata: document columns ON the schema
ALTER TABLE visits
  MODIFY diagnosis_code VARCHAR(10) COMMENT 'ICD-10 3-char code, e.g. A09 (diarrhoea)',
  MODIFY visit_type ENUM('OPD','ANC','IMMUNIZATION','ADMISSION','LAB')
       COMMENT 'OPD=outpatient, ANC=antenatal care';

-- 2. Read it back (the auto data dictionary)
SELECT TABLE_NAME, COLUMN_NAME, DATA_TYPE, COLUMN_COMMENT
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA='medilink' ORDER BY TABLE_NAME, ORDINAL_POSITION;

-- 3. Business glossary
CREATE TABLE glossary (
  term VARCHAR(60) PRIMARY KEY,
  definition VARCHAR(300), owner VARCHAR(100)
);
INSERT INTO glossary VALUES
('Golden Record','Single trusted version of a patient built by MDM survivorship','D. Nakato'),
```

('OPD Attendance','Count of visit rows with visit_type=OPD per period','Head of Clinical Services'),

('Revenue','SUM(fact_visits.total_amount); excludes cancelled visits','Finance Manager');

-- 4. Lineage register

CREATE TABLE data_lineage (

lineage_id INT AUTO_INCREMENT PRIMARY KEY,

source_object VARCHAR(100), target_object VARCHAR(100), transformation VARCHAR(300)

);

INSERT INTO data_lineage VALUES

(NULL,'stg_legacy_a + stg_legacy_b','m_patient','Normalize phone/name, match on phone, survivorship latest-wins'),

(NULL,'visits + billing','fact_visits','Pivot billing amounts to columns, join dims');

-- 5. IMPACT ANALYSIS: what breaks if we change m_patient?

SELECT target_object, transformation FROM data_lineage

WHERE source_object LIKE '%m_patient%' OR target_object LIKE '%m_patient%';

```

***Deliverable: catalog export (dictionary + glossary + lineage) + 1-page impact-analysis example (new-staff onboarding test: a colleague finds the meaning of `diagnosis_code` using only the catalog).***

***PHASE 8 (Week 11): Security & Privacy  Compliance Audit***

```sql

-- 1. Classification register (Rule 6 context)

```
CREATE TABLE data_classification (
  object_name VARCHAR(64) PRIMARY KEY,
  level ENUM('PUBLIC','INTERNAL','CONFIDENTIAL','RESTRICTED')
);
INSERT INTO data_classification VALUES
('m_patient','RESTRICTED'),('visits','RESTRICTED'),
('fact_visits','CONFIDENTIAL'),('dim_clinic','INTERNAL');

-- 2. Hash the National ID at rest (Rule 2) fix the Phase 2 compliance gap
UPDATE stg_legacy_a SET nid = NULL WHERE nid IS NULL;
ALTER TABLE m_patient ADD COLUMN nid_sha CHAR(64);
-- (one-time migration per source record, keyed by crosswalk)
UPDATE m_patient m
JOIN patient_source_links l ON l.patient_key = m.patient_key AND l.source_system='LEGACY_A'
JOIN stg_legacy_a a ON a.src_id = l.source_record_id
SET m.nid_sha = SHA2(a.nid, 256)
WHERE a.nid IS NOT NULL AND a.nid <> '';

-- 3. Masked view for non-clinical staff
CREATE VIEW v_patient_directory AS
SELECT patient_key,
      CONCAT(LEFT(full_name,1),'')   AS name_masked,
      CONCAT('256', RIGHT(phone,3))   AS phone_masked,
      sex, home_district
```

```
FROM m_patient WHERE status='ACTIVE';


-- 4. RBAC (least privilege)
CREATE ROLE 'ml_reception','ml_clinician','ml_analyst','ml_auditor';
GRANT SELECT ON medilink.v_patient_directory TO 'ml_reception';
GRANT SELECT, INSERT, UPDATE ON medilink.m_patient TO 'ml_clinician';
GRANT SELECT ON medilink.fact_visits TO 'ml_analyst';          -- warehouse only, no PII
GRANT SELECT ON medilink. TO 'ml_auditor';

CREATE USER 'agnes'@'%' IDENTIFIED BY 'StrongPass1!';
GRANT 'ml_reception' TO 'agnes'@'%';
SET DEFAULT ROLE ALL TO 'agnes'@'%';

-- 5. Enforce BILLING IMMUTABILITY (Rule 3) in the database itself
DELIMITER //
CREATE TRIGGER trg_billing_immutable
BEFORE UPDATE ON billing FOR EACH ROW
BEGIN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Billing rows are immutable. Insert an adjustment row instead.';
END //
DELIMITER ;

-- 6. Audit trail on the master (Rule 6)
```

```
CREATE TABLE master_audit (
  audit_id INT AUTO_INCREMENT PRIMARY KEY,
  patient_key INT, changed_by VARCHAR(64),
  changed_at DATETIME DEFAULT CURRENT_TIMESTAMP, detail VARCHAR(400)
);
DELIMITER //
CREATE TRIGGER trg_master_audit
AFTER UPDATE ON m_patient FOR EACH ROW
BEGIN
  INSERT INTO master_audit (patient_key, changed_by, detail)
  VALUES (NEW.patient_key, CURRENT_USER(),
        CONCAT('phone:',IFNULL(OLD.phone,'-'),'->',IFNULL(NEW.phone,'-'),
            ' name:',IFNULL(OLD.full_name,'-'),'->',IFNULL(NEW.full_name,'-')));
END //
DELIMITER ;

-- Test the controls:
SELECT  FROM v_patient_directory LIMIT 3;      -- masked ✓
UPDATE billing SET amount=0 WHERE bill_id=1;    -- ERROR: immutable ✓
```

Deliverable: Compliance Audit report completed `compliance_register` (gaps now CLOSED with evidence screenshots) + a DPIA for the new patient-SMS reminder service (data used, lawful basis, risks, mitigations).

PHASE 9 (Weeks 12–13): Analytics & BI the MoH Report

```sql
-- 1. DESCRIPTIVE + ROLL-UP: MoH monthly attendance by district/clinic
SELECT dc.district, dc.clinic_name,
    DATE_FORMAT(f.visit_date,'%Y-%m') AS yr_mo,
    COUNT() AS visits
FROM fact_visits f JOIN dim_clinic dc USING (clinic_sk)
GROUP BY dc.district, dc.clinic_name, yr_mo WITH ROLLUP;

-- 2. DRILL-DOWN: top 5 diagnoses network-wide
SELECT diagnosis_code, COUNT() AS cases,
    DENSE_RANK() OVER (ORDER BY COUNT() DESC) AS rnk
FROM fact_visits
GROUP BY diagnosis_code
ORDER BY rnk LIMIT 5;

-- 3. DIAGNOSTIC: why is revenue down? payment-mode mix per month
SELECT DATE_FORMAT(visit_date,'%Y-%m') yr_mo, payment_mode,
    SUM(total_amount) revenue,
    ROUND(100SUM(total_amount)/SUM(SUM(total_amount)) OVER (PARTITION BY DATE_FORMAT(visit_date,'%Y-%m')),1) AS pct
FROM (SELECT f.visit_date, f.total_amount, 'CASH' AS payment_mode
    FROM fact_visits f) x     -- extend using a payment_mode column on the fact
GROUP BY yr_mo, payment_mode;

-- 4. PREDICTIVE-lite: 3-month moving average of attendance
```

```
SELECT yr_mo, visits,
    ROUND(AVG(visits) OVER (ORDER BY yr_mo ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS ma_3m
FROM (SELECT DATE_FORMAT(visit_date,'%Y-%m') yr_mo, COUNT() visits
    FROM fact_visits GROUP BY yr_mo) t;

-- 5. QUALITY KPI: re-attendance within 30 days (operational metric)
SELECT ROUND(100  COUNT() / NULLIF((SELECT COUNT() FROM
  (SELECT patient_sk, visit_date,
      LEAD(visit_date) OVER (PARTITION BY patient_sk ORDER BY visit_date) nxt
   FROM fact_visits) a WHERE nxt IS NOT NULL),0),1) AS readmit_within_30d_pct
FROM (SELECT patient_sk, visit_date,
        LEAD(visit_date) OVER (PARTITION BY patient_sk ORDER BY visit_date) nxt
    FROM fact_visits) b
WHERE nxt IS NOT NULL AND DATEDIFF(nxt, visit_date) <= 30;
```

Deliverable: dashboard SQL pack + one chart per query (Power BI/Tableau on the warehouse) + 1-page finding presented to a "management" role-play.

PHASE 10 (Weeks 14–15): Automation, Trends & Final Strategy Assessment 6 (20%)

```sql
-- Automation (DataOps spirit): nightly DQ scorecard log + lifecycle enforcement
CREATE TABLE dq_scorecard_log (
  run_date DATE PRIMARY KEY, phone_complete DECIMAL(5,2),
```

```
  dob_complete DECIMAL(5,2), dupes INT
);
DELIMITER //
CREATE EVENT ev_nightly_dq
ON SCHEDULE EVERY 1 DAY
DO INSERT INTO dq_scorecard_log
   SELECT CURDATE(), phone_complete_pct, dob_complete_pct, potential_dupes
   FROM v_dq_scorecard;
DELIMITER ;

-- Retention automation (demonstration window shortened; production = 10 years)
DELIMITER //
CREATE EVENT ev_archive_old_visits
ON SCHEDULE EVERY 1 DAY
DO UPDATE visits SET status='ARCHIVED'
   WHERE status='ACTIVE' AND visit_date < NOW() - INTERVAL 10 DAY;
DELIMITER ;
```

Final Integrated EDM Strategy document (the 20% deliverable) required sections:

1. Executive summary (business problem → what the platform delivers)
2. Governance: charter, RACI, policies, compliance register (before/after evidence)
3. Architecture diagram: sources → staging → master (MDM) → warehouse → BI
4. Data quality: metrics achieved vs targets (e.g., phone 71% → 100%)
5. MDM: matching/survivorship rules + review workflow
6. Metadata & lineage: catalog extracts
7. Security & privacy: RBAC matrix, masking, DPIA, breach playbook

8. Analytics: the MoH report pipeline
9. Roadmap: cloud migration proposal (managed MySQL vs on-prem), where data mesh would apply (each clinic as a data domain), AI candidates (auto-coding diagnoses)
10. Individual reflection + contribution log

Demo script (rehearse, 15 min):

1. Show raw dirty staging → run scorecard → cleanse → scorecard improves.
2. Three sources with the same patient → MDM run → one golden record + crosswalk.
3. ETL load → MoH report query returns in milliseconds.
4. Attempt to edit billing → blocked (governance enforced in the database).
5. Query as reception (masked) vs clinician (full).
6. `SHOW EVENTS` retention and DQ automation running.
7. New analyst finds `diagnosis_code` meaning using only the catalog.

Sample defense questions:

1. Why match on phone and not name? What's the failure mode of each?
2. Which survivorship decision would a steward challenge, and why is `mdm_review_queue` better than silent merging?
3. Where exactly is each governance policy enforced technically? Name the object (trigger/constraint/role).
4. Why is `fact_visits` a separate schema from `visits`? Why pivot billing there?
5. Your DQ target was 97% phone completeness how is it measured and who is accountable?
6. Which GDPR/DPA principle does hashing the National ID serve? Is hashed data still personal data?
7. If a new clinic joins the network, what must they do before their data is trusted? (Rule 9 + catalog)

8. What would break first at 10× volume staging loads, matching, or reports and what is the fix?

DELIVERABLES → COURSE ASSESSMENT MAP

Phase	Artifact	Course assessment
2	Governance Framework + working register/RACI/policies	Assessment 1 (20%)
4	Data Quality Report + scorecard view	Assessment 2 (15%)
5	MDM Plan + golden records + review queue	Assessment 3 (15%)
6	ETL workflow doc + populated warehouse	Assessment 5 (10%)
7–8	Catalog + compliance audit + DPIA	Week-12 audit evidence
9	Dashboard SQL + charts	feeds Final
10	Integrated EDM Strategy + presentation	Assessment 6 (20%)

One-line summary for students

You are not "doing data management assignments" you are the EDM team that rescued a merged clinic network: you inventoried and governed the data (P1–2), modeled the master (P3), cleaned and monitored it (P4), gave every patient one identity (P5), built the warehouse that reports to the Ministry (P6), documented everything so knowledge survives staff turnover (P7), locked down privacy by law (P8), turned data into management insight (P9), and automated it for the future (P10) the complete lifecycle of real enterprise data work.